# Renze backend v3 — C9 replay demo and backend design

Nov 10, 2026 · @Yuhan Chiang

## Summary

The C9 demo replays one cleaning cycle (2026-04-27 → 05-18) in about 4.5 minutes: a Python replay agent drops one 15-minute CSV packet per file into an inbox folder, and a Python backend reads it, scores every minute with both RUL models, and commits to SQLite; Node only reads SQLite and pushes to the dashboard. Part 1 specifies the reading and writing code to build first. Part 2 holds the rest of the backend. Open items are listed at the end.

Decisions converged on 2026-11-10:

| Topic | Decision |
|---|---|
| Demo scope | One cycle, 2026-04-27 → 05-18 of `series6_cycle_9_converted.npy`; live talk, 3–5 min run |
| Process split | Replay agent (Python) → inbox folder → Python backend (ingest, 1-min samples, scoring, decision, SQLite) → Node API (read-only) → Next.js |
| Packet | One file per clock quarter-hour (:00, :15, :30, :45) = 180 rows × 5 s |
| 1-min sample | Mean of the 12 rows of 5 s in that minute |
| Models | Pipe and HXG RUL LSTMs as trained (15 × 11 input); both receive the same 11-sensor copy of the data |
| Inference | Direct call or `tf.function`, never `model.predict()`; windows batched per packet |
| Scoring cadence | One prediction per minute (the smoothing cadence); 1440-prediction causal rolling mean for display |
| Critical rule | Rolling 15-min mean of VCB2-Power < 300 kW → RUL output shown as Critical; checked continuously |

| Topic | Decision |
|---|---|
| Offline display | Downtime shown as "plant offline"; agent fast-forwards offline packets |
| Dashboard refresh | About 4–5 updates per second |

Changes from backbone v2: the HTTP push/ack between client and server is replaced by a shared folder; gap check, bundling and database writes move from Node into Python; the scoring unit changes from a 10-min bundle to a 1-min sample with a 15-minute window.

# Part 1 — Demo reading and writing

Two Python programs share one folder and never call each other: the replay agent writes one packet file per quarter-hour of data, and the backend reader consumes the files strictly in time order. The file format is the contract between them, so the same reader later serves the production folder-ingest mode.

## 1.1 Source data

| Item | Value |
|---|---|
| File | `08-Data/series6_cycle_9_converted.npy` |
| Shape and type | 1,036,800 × 50, float64, about 415 MB; one row every 5 s, no gaps, no NaN |
| Columns | 0 = epoch seconds (UTC); 1–49 = `RAW_SENSOR_COLS` from notebook 8-6 v3, `PT104S` last |
| Demo window | 2026-04-27 00:00 → 05-18 00:00 UTC: 362,880 rows = 2,016 packets |
| Operating span | 04-27 16:55 → 05-11 09:25 UTC: 328.5 h, about 1,314 packets |
| Cycle close (VCB2 < 10 kW for 48 h) | 05-13 09:46:15 UTC |

All times are UTC as stored in column 0; Taiwan local time is UTC+8 (see open items).

## 1.2 Folder layout

```
demo_root/
  inbox/              agent publishes finished packets here
  inbox/.tmp/         agent writes here first, then renames
  inbox/processed/    reader moves committed packets here
  inbox/rejected/     reader moves invalid packets here
  raw/                reader's raw copy (copy 2), one CSV per data day
  control/            agent_status.json and the EOS marker
  renze.db            SQLite database (Part 2)
```

## 1.3 Packet file contract

- **Name:** `renze_YYYYMMDDTHHMMZ.csv`, the quarter's start in UTC, for example `renze_20260427T1645Z.csv`. Alphabetical order equals time order.
- **Boundaries:** clock quarter-hours (:00, :15, :30, :45); a packet holds the 180 rows with `ts` = q, q+5, …, q+895.
- **Columns:** header line, then `ts` (integer epoch seconds) followed by the 49 sensor names in `RAW_SENSOR_COLS` order. `PT104S` is kept in the file so it matches what the plant delivers; the reader drops it.
- **Values:** UTF-8, `.` as decimal point, no thousands separator, 9 significant digits, `\n` line ends. About 90 KB per file.
- **Atomic publish:** the agent writes `inbox/.tmp/<name>.part`, then renames it into `inbox/` with `os.replace`. The reader ignores `.tmp/` and any name that does not match the pattern, so it never sees a half-written file.

## 1.4 Replay agent (writer)

1. Open the source with `np.load(..., mmap_mode='r')` and find the window's row range by binary search on column 0.
2. Walk the quarter-hours from the window start; slice the 180 rows of each.
3. Classify each packet for pacing only: offline if its mean VCB2-Power is below 10 kW, otherwise operating. The backend makes its own Critical and offline decisions.
4. Wait for the packet's scheduled wall time. The schedule runs on a monotonic clock (`next_due += 1 / rate`), so delays never accumulate.
5. Write the `.part` file, rename it into `inbox/`, and update `control/agent_status.json` (sequence number, quarter, state, wall time).
6. After the last packet, write `control/EOS`.

| Setting | Default | Purpose |
|---|---|---|
| `source`, `start`, `end` | the file and window in 1.1 | What to replay |
| `inbox` | `demo_root/inbox` | Where to publish |
| `op_rate` | 5 packets/s | Operating pace: 75 data-minutes per second, 4,500× real time |
| `offline_rate` | 100 packets/s | Fast-forward through downtime |
| `duration` | none | If set, overrides `op_rate` so the run lasts this many seconds |
| `drop_quarters` | empty | Quarters never written, to test gap handling |
| `late_quarters` | empty | Quarters written N packets late, to test late arrivals |

At the defaults the run takes about 1,314 / 5 ≈ 263 s operating plus 702 / 100 ≈ 7 s offline, roughly 4.5 minutes. `op_rate` 7.5 gives about 3 minutes.

## 1.5 Backend reader

1. Poll `inbox/` every 100 ms with `os.scandir`, keep names matching the pattern, and sort them.
2. Process strictly in quarter order. If the next expected quarter is missing while a later file is waiting, wait up to `gap_wait_s` (default 2 s wall time), then record that quarter as missing with a "data gap" message and continue. A file for a quarter already passed is stored in the raw copy and logged as late, but not scored.
3. Parse the CSV with numpy and validate it: header equals the contract, 180 rows, `ts` equals q + 5k, every value finite. A failure moves the file to `rejected/`, posts a message, and counts the quarter as missing.
4. Append the 180 rows to `raw/YYYYMMDD.csv` (the reader's raw copy).
5. Build 15 one-minute samples: drop `PT104S`, reshape to 15 × 12 × 48, and take the mean over the 12 rows. Each sample is stamped with its minute's start.
6. Pass the 15 samples to the processor (Part 2) and commit everything for this packet in one SQLite transaction.
7. Move the file to `processed/`. After `EOS`, with nothing left to read, post "replay finished".

Re-running a packet is safe: a quarter already in the `packets` table is skipped, so a crash between commit and move only repeats the move. Duplicate raw rows from such a crash are removed by timestamp at cycle close.

### 1.6 Build order and checks

1. `packet_io.py`, shared by both programs: quarter ↔ file name, CSV write and read, validation, 1-min means. The contract lives in one place.
2. `replay_agent.py`, with a dry-run mode that prints the schedule and the expected run time without writing.
3. `reader.py` with a stub processor that only writes `packets` and `minutes` to SQLite.
4. Round-trip check: replay the window and compare every stored 1-min mean with the same mean computed straight from the `.npy`; relative difference below 1e-6.
5. Gap and late check: run with one `drop_quarters` and one `late_quarters` entry and confirm the messages and table states.

Scoring, the database schema and the API come after this works (Part 2).

## Part 2 — Other backend designs

The processor turns each packet's 15 one-minute samples into scores, statuses and, once per operating day, a maintenance decision, and commits them with its own state in one transaction; Node reads the result and never writes model data.

### 2.1 Per-minute processing

For each of the packet's 15 samples, in time order:

1. **VCB2 rolling mean:** `roll15` = mean VCB2-Power over the trailing 15 one-minute samples, carried across packets in the worker state.
2. **Status:** Critical if `roll15` < 300 kW. Otherwise Warm-up while the window buffer holds fewer than 15 samples. Otherwise Normal.
3. **Window buffer:** if the sample's own VCB2-Power is at least 300 kW, its 11 model features enter a 15-sample FIFO. This mirrors training, where rows below 300 kW and downtime were removed before windows were cut.
4. **Window:** a Normal sample that entered the buffer yields one 15 × 11 window to score.

Then, per packet:

5. **Batched inference:** stack the packet's windows (at most 15). Scale them with the pipe scaler and call the pipe model once; scale the same raw windows with the HXG scaler and call the HXG model once. Both models get the same 11-sensor copy of the data;

each keeps its own MinMaxScaler.

6. **Smoothing:** per model, a causal FIFO of the last 1,440 raw scores (24 h at one per minute). The displayed value is the raw score until the cache is full, then the cache mean.
7. **RUL:** minutes = score × 10,080 / 100; operating days = minutes / 1,440. A score of 100 is shown as "7 days or more".
8. **Irregular-failure clock:** `OperatingClock` from `renze_maint_core.py` is updated with each smoothed score pair.
9. **Operating-minute counter:** +1 for every sample with VCB2-Power of at least 300 kW; it drives the daily decision.

Model set-up: rebuild each model with `build_model(15, 11, 32)` and `load_weights`, because the Lambda layer blocks `load_model` in TF 2.12. Wrap the forward pass in `tf.function` with input signature `[None, 15, 11]` and `training=False`, run one warm-up call on zeros at start-up, and limit TensorFlow to one or two threads. Never call `model.predict()`.

## 2.2 Daily decision

- **Trigger:** every 1,440 operating minutes since cycle start; about 13 decisions in the C9 window.
- **Inputs:** that operating day's means of TT203, PT100, PT105, LIT101, PT106 and LV102 from the 1-min samples, plus the smoothed RUL of each component in operating days.
- **Calls:** `recovery_kwh` three times — joint with min(RUL pipe, RUL HX), pipe with `SHARE_PIPE`, HX with `SHARE_HX` — then FIT × kWh − C per option, and the recommendation.
- **Kept from v2:** the RUL override (a component whose RUL reaches 0 within the 72 h window → maintain immediately), the irregular-failure alert, and 0 TWD on the first day after restart. Every call is logged in `decisions`.
- **Known limit:** model RUL is capped at 7 days, while the recovery function was trained on RUL up to about 18 days (production guide v0).

## 2.3 Cycle lifecycle

- **Start:** the replay start, or the first operating minute after a close. Reset the window buffer, both smoothing caches, `OperatingClock`, the operating-minute counter and the day index.
- **Close:** VCB2-Power below 10 kW on every 5 s row for 48 h of data time. In the C9 window this fires at 2026-05-13 09:46:15 UTC.

- **On close:** post a message, export the cycle (minutes, scores, decisions) to CSV, de-duplicate the raw copy by timestamp and check its row count against packets × 180, then move `raw/` and `processed/` into `raw/cycle_<id>/`.
- **After close:** packets keep arriving until 05-18; they are stored and shown as plant offline, with no scoring, until the next operating minute starts a new cycle.

## 2.4 SQLite schema

Pragmas: `journal_mode=WAL`, `synchronous=NORMAL`, `busy_timeout=5000`, `foreign_keys=ON`. One transaction per packet writes its `packets` row, 15 `minutes`, 15 `scores` and `worker_state`, plus `op_days`, `decisions` and `messages` when they occur.

| Table | Key | Columns | Rows in the C9 window |
|---|---|---|---|
| `cycles` | `cycle_id` | started_ts, closed_ts, close_reason, export_path | 1–2 |
| `packets` | `quarter_ts` | cycle_id, file_name, status (ok, missing, late, rejected), n_rows, received_wall, committed_wall | 2,016 |
| `minutes` | `ts` | cycle_id, the 48 canonical sensors, vcb2_roll15 | 30,240 |
| `scores` | `ts` | cycle_id, status (Normal, Warm-up, Critical), pipe_raw, pipe_smooth, hx_raw, hx_smooth, pipe_rul_days, hx_rul_days, model_version | 30,240 |
| `op_days` | `cycle_id, day_idx` | start_ts, end_ts, the six decision-sensor means | 13 |
| `decisions` | `cycle_id, day_idx` | ts, RULs, recovery kWh (joint, pipe, HX), net TWD per option, recommendation, override, alerts, model and JSON versions, shares | 13 |
| `worker_state` | `id = 1` | cycle_id, last_quarter_ts, window buffer (15 × 11), two smoothing caches (≤ 1,440 each), VCB2 trail (15), operating minutes, day index, `OperatingClock` JSON | 1 |
| `messages` | `msg_id` | ts_data, ts_wall, level, type, text, payload JSON, acked | tens |

| Table | Key | Columns | Rows in the C9 window |
|---|---|---|---|
| `shutdown_log` | `entry_id` | ts_wall, ts_data, author, reason, notes | user entries |

Because `worker_state` commits with each packet's results, a restart resumes from the last committed quarter with nothing counted twice.

## 2.5 Node API and dashboard

- **Reading:** `better-sqlite3` on a read-only connection; WAL lets it read while Python writes.
- **Push:** every 200 ms, select scores, messages and decisions newer than the last ones sent and send them as one Server-Sent Events batch ( `tick` , `message` , `decision` , `cycle` , `replay` ). At 5 packets/s each batch carries about one packet.
- **REST:** `GET /api/state` (snapshot for a newly opened browser), `/api/series?from&to` , `/api/messages` , `/api/decisions` , `/api/cycles` , `/api/export/:cycle_id` ; `POST /api/shutdown-log` is the only write Node makes.
- **Dashboard:** smoothed RUL for pipe and HX with a status band (Critical, Warm-up, offline), VCB1 and VCB2 power, the message list, popups for decisions, Critical, irregular failure and cycle close, and download buttons.

## 2.6 Messages

| Type | Trigger | Level |
|---|---|---|
| `critical_on` / `critical_off` | `roll15` crosses 300 kW (transitions only) | critical / info |
| `irregular_failure` | `OperatingClock` raises its alert | critical |
| `model_error` | inference fails for a packet | critical |
| `data_gap` | an expected quarter is still missing after `gap_wait_s` | warning |
| `late_packet` | a file arrives for a quarter already passed | warning |
| `rejected_packet` | a file fails validation | warning |
| `sensor_out_of_range` | a model feature lies outside its scaler's training range | warning |
| `decision` | a daily decision is recorded | info, popup |

| Type | Trigger | Level |
|---|---|---|
| `cycle_closed` | the close rule fires | info, popup |
| `replay_finished` | `EOS` processed with nothing left | info |

### 2.7 Load and settings

At the default 4.5-minute pace the C9 window means 2,016 packets (about 1,314 operating), about 2,628 model calls, about 19,700 windows per model, 2,016 SQLite transactions and 13 decisions. Each operating packet has about 200 ms; reading, parsing and averaging are expected to take a few ms, so the margin goes to the two batched model calls and the commit. To be measured on the Mac before the demo: time per model call and per transaction at this pace.

## Open items

None of these blocks Part 1; items 1, 2 and 4 must be settled before the scoring code.

- [ ] **Critical vs plant offline:** under the 300 kW rule, the 7-day shutdown also reads as Critical. Proposal: plant offline when `roll15` < 10 kW, Critical when 10 ⩽ `roll15` < 300 kW.
- [ ] **Time zone:** the window and all stored times are UTC from the data; should the dashboard show Taiwan time (UTC+8)? Quarter-hour alignment is the same in both.
- [ ] **Late packets:** v1 stores them raw but does not score them. Rescoring would mean replaying state from that quarter onward.
- [ ] **Critical and the decision:** should a Critical period skip the day's decision, force a recommendation, or leave it unchanged?
- [ ] **Averaging check:** backend 1-min samples are means of 12 rows; cycle 8 training samples were 30 s means taken every other row. Check by replaying cycle 8 and comparing scores with notebook 06.
- [ ] **RUL range:** the model caps at 7 days, the recovery function was trained to about 18 days (production guide v0).
- [ ] **Out-of-range margin:** how far outside the scaler's training range a feature may go before `sensor_out_of_range` fires.
- [ ] **HXG notebook names list:** it lists 16 names while the 11 indices used match the pipe list; a comment fix only, the model stays as trained.